



รายงานโครงงานวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology)

ภาคเรียนที่ 1 ปีการศึกษา 2569

โครงงานระบบจำลองห้องเล่นและวิเคราะห์เด็คการ์ดแบบเรียลไทม์บนคลาวด์ Serverless

(Disney Lorcana PlayLab Cloud: Real-Time Serverless Card Game Simulator)

กลุ่ม G21

รายงานความก้าวหน้าโครงการ (Stage 2: บทที่ 1 - 3 และส่วนขยายระบบคลาวด์)

คณะผู้จัดทำ (กลุ่ม G21):

- | | |
|----------------------------------|-----------------------|
| 1. นายชยุต บุญวัฒน์ | รหัสนักศึกษา 67070032 |
| 2. นายธนัทภัทร พรหมทอง | รหัสนักศึกษา 67070069 |
| 3. นายภูริ ประชาสุขสิน | รหัสนักศึกษา 67070137 |
| 4. นางสาววรรณนิศา อมรวงศ์ไพบุลย์ | รหัสนักศึกษา 67070155 |
| 5. นายอสิธรา พุ่มดอกไม้ | รหัสนักศึกษา 67070199 |
| 6. นายวรวิชณ์ คงทอง | รหัสนักศึกษา 67070275 |

อาจารย์ประจำวิชา:

1. ดร. ธนเทพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ อินทมิตรโสภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

สิงหาคม 2569

บทคัดย่อ

โครงการ Disney Lorcana PlayLab Cloud นำเสนอการวิจัยและพัฒนาระบบจำลองการเล่นและการวิเคราะห์สมมูลเด็คการ์ดสะสม (Trading Card Game: TCG) บนเว็บแอปพลิเคชันแบบเรียลไทม์ โดยประยุกต์ใช้สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% Serverless Architecture) บน Amazon Web Services (AWS) ภายใต้ข้อจำกัดด้านงบประมาณฟรีของ AWS Free Tier และการทำงานบนสภาพแวดล้อมสิทธิ์จำกัดของ AWS Academy Learner Lab โดยมีวัตถุประสงค์เพื่อแก้ปัญหาค่าใช้จ่ายเซิร์ฟเวอร์เปิดค้าง และความซับซ้อนในการฝึกซ้อมการเล่นการ์ด

ระบบได้รับการออกแบบแบ่งเป็น 4 เลเยอร์สำคัญ ได้แก่ Presentation Layer พัฒนาด้วย React 19 SPA และ Tailwind CSS v4, API & Routing Layer ใช้ AWS API Gateway ทั้งในรูปแบบ HTTP API และ WebSocket API สำหรับการสื่อสารสองทิศทางที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที, Compute Layer ใช้ AWS Lambda Microservices และ Persistence Layer จัดเก็บข้อมูลบน Amazon DynamoDB ร่วมกับระบบคิวข้อความ Amazon SQS

นอกจากนี้ รายงานฉบับนี้ได้นำเสนอกลยุทธ์ทางวิศวกรรมในการรับมือกับความล้มเหลวของ API ภายนอกแบบ 4 ระดับ (4-Tier Fallback Strategy) แนวทางการสเกลและรองรับ Concurrency ของ WebSockets การออกแบบระบบตามกรอบ AWS Well-Architected Framework 4 เสาหลัก (Reliability, Availability, Security, Cost Optimization) และการประเมินคุณภาพระบบผ่าน Master QA Test Matrix ซึ่งผลการทดสอบจริงยืนยันว่าระบบมีความหน่วงในการส่งข้อมูลระหว่างผู้เล่นเพียง 52-82 มิลลิวินาที และมีค่าใช้จ่ายโครงสร้างพื้นฐาน 0.00 ดอลลาร์สหรัฐต่อเดือน

คำสำคัญ: สถาปัตยกรรมไร้เซิร์ฟเวอร์, เว็บซ็อกเก็ตแบบเรียลไทม์, อเมซอนเว็บเซอร์วิส, ดิสนีย์ ลอร์คานา, การทดสอบคุณภาพระบบ, AWS Learner Lab

Abstract

The Disney Lorcana PlayLab Cloud project presents the architectural design, implementation, and quality assurance of a real-time web-based Trading Card Game (TCG) simulator and automated deck analytics platform. Built upon a 100% AWS Serverless Architecture, the system operates strictly within the zero-cost tier of the AWS Free Tier and complies with the permission-restricted environment of AWS Academy Learner Lab.

The system is architected into four decoupled layers: (1) Presentation Layer utilizing React 19 Single Page Application with Tailwind CSS v4, (2) API and Routing Layer leveraging AWS API Gateway for both RESTful HTTP and persistent bi-directional WebSocket connections, (3) Compute Layer powered by AWS Lambda Microservices, and (4) Persistence Layer using Amazon DynamoDB on-demand NoSQL tables combined with Amazon SQS messaging queues.

Key engineering contributions include a 4-Tier External API Fallback Strategy that guarantees 100% uptime even during third-party service outages, high-concurrency WebSocket scalability techniques utilizing DynamoDB TTL auto-cleanup and Optimistic UI updates, comprehensive architectural alignment with the AWS Well-Architected Framework, and engineering workarounds for AWS Learner Lab restrictions. Empirical benchmark results demonstrate an average round-trip synchronization latency of 52–82ms with zero infrastructure monthly cost.

Keywords: Serverless Architecture, Real-Time WebSockets, Amazon Web Services, Disney Lorcana, Quality Assurance, AWS Learner Lab

กิตติกรรมประกาศ

โครงการ Disney Lorcana PlayLab Cloud สำเร็จลุล่วงได้ด้วยความรู้และการให้คำปรึกษาอย่างดียิ่งจากคณาจารย์ประจำวิชาเทคโนโลยีกลุ่มเมฆ (Cloud Technology) คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ทั้ง 3 ท่าน ได้แก่ ดร. ธนานพ ทองถาวร, ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส และ ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์ ผู้ซึ่งได้ถ่ายทอดองค์ความรู้ คำแนะนำเชิงสถาปัตยกรรม และการประเมินคุณภาพระบบคลาวด์ที่เป็นประโยชน์อย่างยิ่งตลอดระยะเวลาการดำเนินโครงการ

คณะผู้จัดทำขอขอบพระคุณคณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง สำหรับการสนับสนุนสิ่งอำนวยความสะดวก และ AWS Academy สำหรับแพลตฟอร์ม Learner Lab ที่เปิดโอกาสให้นักศึกษาได้ฝึกปฏิบัติการบนโครงสร้างพื้นฐานคลาวด์ระดับโลกอย่างแท้จริง

ท้ายที่สุดนี้ คณะผู้จัดทำขอขอบคุณเพื่อนร่วมกลุ่ม G21 ทุกท่านที่มีความมุ่งมั่น ทุ่มเท และร่วมมือกันแก้ไขปัญหาทางวิศวกรรมจนทำให้โครงการนี้สำเร็จลุล่วงได้ตามเป้าหมายอย่างมีประสิทธิภาพ

คณะผู้จัดทำ (กลุ่ม G21)

สิงหาคม 2569

สารบัญ

บทคัดย่อ	i
Abstract	ii
กิตติกรรมประกาศ	iii
1 บทนำ (Introduction)	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของระบบ (System Scope)	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Related Theories & Technologies)	3
2.1 สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์ (AWS Serverless Architecture)	3
2.2 สถาปัตยกรรมการสื่อสารแบบสองทิศทางเรียลไทม์ (WebSockets Protocol)	3
2.3 เทคโนโลยีฝั่งผู้ใช้งาน (Frontend Engineering & State Architecture)	3
2.4 โมเดลโดเมนเกมการ์ดสะสม Disney Lorcana	4
2.5 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum และการดำเนินงาน Stage 2	4
3 การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)	5
3.1 ข้อกำหนดความต้องการของระบบ (Requirements)	5
3.1.1 Functional Requirements (FR)	5
3.1.2 Non-Functional Requirements (NFR)	5
3.2 สถาปัตยกรรมระบบโดยรวม (End-to-End Serverless Architecture)	5
3.3 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)	7
3.4 การออกแบบการจัดการ API ภายนอกและการป้องกันความล้มเหลว (4-Tier Fallback Strategy)	7
3.5 การออกแบบการสเกลและรองรับ WebSockets Concurrency	8
3.6 การออกแบบตามกรอบ AWS Well-Architected Framework	9
3.7 การวิเคราะห์ข้อจำกัดของ AWS Learner Lab และแนวทางแก้ไขเชิงวิศวกรรม	9

3.7.1	ผลการตรวจสอบโครงสร้างพื้นฐานจริงบนคลาวด์ (Live Infrastructure Verification)	9
4	การพัฒนาระบบและผลการทดสอบคุณภาพ (Implementation & Quality Assurance)	11
4.1	รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation)	11
4.2	รายละเอียดการพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Microservices)	15
4.3	แผนและผลการทดสอบคุณภาพระบบ (Quality Assurance & Test Verification)	15
5	สรุปผลการดำเนินงานและแผนงานในอนาคต (Conclusions & Future Roadmap)	17
5.1	สรุปผลการดำเนินงานความก้าวหน้า Stage 2	17
5.2	แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)	17
5.3	บทเรียนที่ได้รับและการปรับปรุงกระบวนการทำงาน (Retrospective)	17
	บรรณานุกรม (References)	19
	ภาคผนวก (Appendix)	20

สารบัญตาราง

3.1	โครงสร้างตารางข้อมูล Amazon DynamoDB (NoSQL Schema Design)	7
4.1	ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)	16

สารบัญรูปภาพ

3.1	ชุดไอคอนบริการคลาวด์ AWS ที่ใช้ในโครงงาน (AWS Architecture Icons – Official)	6
3.2	สถาปัตยกรรมระบบคลาวด์แบบ Serverless 4 เลเยอร์ (AWS Serverless Architecture)	6
3.3	กลยุทธ์การป้องกันความล้มเหลวของ API ภายนอกแบบ 4 ระดับ (4-Tier Multi-Layer Fallback)	7
3.4	ผังการสื่อสารและซิงค์ข้อมูลกระดานเกมแบบเรียลไทม์ผ่าน AWS API Gateway WebSockets	8
4.1	หน้าแรกของเว็บแอปพลิเคชันแบบเต็มหน้า (Landing Page Full View – Disney Lorcana PlayLab)	12
4.2	ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud (ส่วนที่ 1)	13
4.3	ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud (ส่วนที่ 2)	14
4.4	ภาพหน้าจอหน้าระบบคู่มือการเล่นและระบบวิเคราะห์สถิติได้ค	14

บทที่ 1

บทนำ (Introduction)

1.1 ความเป็นมาและความสำคัญของปัญหา

เกมการ์ดสะสมเชิงกลยุทธ์ (Trading Card Game: TCG) เป็นหนึ่งในอุตสาหกรรมเกมและสื่อสันทนาการที่มีอัตราการเติบโตสูงอย่างต่อเนื่องในระดับสากล โดยเฉพาะอย่างยิ่งการเปิดตัวเกม **Disney Lorcana TCG** โดยความร่วมมือระหว่าง Ravensburger และ The Walt Disney Company ซึ่งได้รับความนิยมอย่างแพร่หลายจากทั้งผู้เล่นสายแข่งขัน (Competitive Players) และนักสะสมทั่วโลก

อย่างไรก็ดี ในบริบทของการฝึกซ้อมและการเข้าถึงเกมการ์ด ผู้เล่นส่วนใหญ่ยังคงประสบกับข้อจำกัดหลัก 3 ประการ:

1. **ต้นทุนการจัดการจริงและข้อจำกัดทางกายภาพ:** การ์ดหายากมีราคาสูงในตลาดรอง และการนัดพบเพื่อเล่นเกมจริงมีข้อจำกัดด้านสถานที่และเวลา
2. **ภาระต้นทุนเซิร์ฟเวอร์และประสิทธิภาพของระบบดั้งเดิม:** ระบบจำลองเกมการ์ดออนไลน์ส่วนใหญ่ในอดีตสร้างขึ้นบนสถาปัตยกรรม Virtual Machine (VM) หรือ Dedicated Server ซึ่งต้องเปิดทำงานค้างไว้ตลอด 24 ชั่วโมง ส่งผลให้มีค่าใช้จ่ายโครงสร้างพื้นฐานสูงแม้ในช่วงเวลาที่ไม่มีผู้ใช้งาน (Zero-Traffic Idle Cost)
3. **ความหน่วงในการส่งข้อมูล (High Latency) จากการใช้ HTTP Polling:** ระบบเว็บแบบดั้งเดิมมักใช้การดึงข้อมูลเป็นระยะ (Polling) ซึ่งสร้างภาระแก่เซิร์ฟเวอร์และทำให้การอัปเดตกระดานไม่ทันเวลา ส่งผลต่อประสบการณ์การเล่นเกมเชิงกลยุทธ์

เพื่อแก้ไขปัญหาดังกล่าว คณะผู้จัดทำจึงได้เริ่มโครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** โดยนำสถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% AWS Serverless Architecture) ร่วมกับเทคโนโลยีเว็บสมัยใหม่ มาใช้ในการพัฒนาระบบจำลองกระดานเล่นการ์ดและระบบวิเคราะห์เด็คอัจฉริยะ ที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที สามารถขยายขนาดรองรับผู้เล่นได้โดยอัตโนมัติ และควบคุมค่าใช้จ่ายโครงสร้างพื้นฐานให้อยู่ภายใต้งบประมาณฟรีของ AWS Free Tier (0.00 ดอลลาร์สหรัฐ) ได้อย่างสมบูรณ์

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานเล่นเกมการ์ด Disney Lorcana แบบเรียลไทม์บนเว็บเบราว์เซอร์ที่มีความหน่วงต่ำ (<100ms) ผ่านช่องทางสื่อสารสองทิศทาง WebSockets
2. เพื่อประยุกต์ใช้สถาปัตยกรรม AWS Serverless (AWS Lambda, Amazon DynamoDB, AWS API Gateway) ในการสร้างระบบ Microservices ที่ประหยัดพลังงาน ยืดหยุ่น และปรับขนาดตามโหลดอัตโนมัติ
3. เพื่อพัฒนาระบบจัดการเด็คการ์ด (Deck Builder) และระบบประมวลผลวิเคราะห์สมมูลเด็คแบบ Asynchronous ผ่าน Amazon SQS
4. เพื่อศึกษาและวางมาตรการด้านความเชื่อถือได้ (Reliability), ความพร้อมใช้งาน (Availability), ความปลอดภัย (Security) และการควบคุมต้นทุน (Cost Optimization) ตามกรอบ AWS Well-Architected Framework
5. เพื่อศึกษาและแก้ไขปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อมสิทธิ์จำกัดของ AWS Academy Learner Lab (งบประมาณ 50 ดอลลาร์สหรัฐ)

1.3 ขอบเขตของระบบ (System Scope)

ระบบครอบคลุมฟังก์ชันการทำงานหลัก 9 ด้าน (LC-01 ถึง LC-09) ได้แก่:

- ระบบยืนยันตัวตนและความปลอดภัย (Custom Auth ด้วย bcrypt และ JWT Token)
- ระบบจัดการเหตุการณ์บน DynamoDB รองรับการบันทึก ดึงข้อมูล แก้ไข และลบได้
- ระบบจำลองการเปิดช่องการ์ด Booster Pack และระบบตรวจสอบการ์ด 3 มิติ (3D Inspector)
- ระบบจำลองกระดานและห้องเล่นเรียลไทม์ผ่าน AWS API Gateway WebSocket
- ระบบวิเคราะห์เหตุการณ์แบบ Asynchronous ผ่าน Amazon SQS
- ระบบติดตามประสิทธิภาพและตรวจสอบข้อผิดพลาดผ่าน Amazon CloudWatch

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นมีเครื่องมือฝึกซ้อมและวิเคราะห์เหตุการณ์เสมือนจริงที่ใช้งานได้ฟรีผ่านเว็บเบราว์เซอร์
2. ได้รับความรู้เชิงลึกในการออกแบบสถาปัตยกรรม Serverless Microservices บน AWS
3. ได้แนวทางปฏิบัติที่เป็นรูปธรรมในการรับมือกับความล้มเหลวของ API ภายนอกและการสเกล WebSockets
4. ลดภาระต้นทุนด้านโครงสร้างพื้นฐานลงได้ 100% บน AWS Free Tier

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Related Theories & Technologies)

2.1 สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์ (AWS Serverless Architecture)

สถาปัตยกรรม Serverless เป็นแนวทางการพัฒนาซอฟต์แวร์ที่นักพัฒนาไม่ต้องบริหารจัดการ Provisioning, Patching หรือ Scaling ของเครื่องเซิร์ฟเวอร์ โดยระบบจะคิดค่าบริการตามปริมาณการเรียกใช้งานจริงในระดับมิลลิวินาที ซึ่งบริการหลักที่ใช้งานในโครงงานนี้ประกอบด้วย:

- **AWS Lambda:** ระบบประมวลผลฟังก์ชันแบบ Event-Driven (FaaS) ที่รันโค้ด Node.js 20.x แบบ On-Demand
- **Amazon API Gateway:** ระบบบริหารจัดการ API แบบรวมศูนย์ รองรับทั้ง HTTP API (REST) และ WebSocket API (WSS)
- **Amazon DynamoDB:** ฐานข้อมูล NoSQL แบบ Key-Value / Document ที่มีความหน่วงในระดับ Single-Digit Millisecond
- **Amazon SQS:** ระบบคิวข้อความแบบกระจายตัวที่ช่วยจัดการฟังก์ชันระหว่าง Microservices (Decoupling)
- **Amazon CloudWatch:** ระบบตรวจสอบ บันทึก Log และแจ้งเตือนค่าใช้จ่ายแบบเรียลไทม์

2.2 สถาปัตยกรรมการสื่อสารแบบสองทิศทางเรียลไทม์ (WebSockets Protocol)

โพรโทคอล WebSocket (RFC 6455) เป็นมาตรฐานการสื่อสารสองทิศทางแบบ Full-Duplex ผ่านช่องทาง TCP เดียวกัน ทำให้ไคลเอนต์และเซิร์ฟเวอร์สามารถส่งผ่านข้อความหากันได้ทันทีโดยไม่ต้องเปิด Connection ใหม่ทุกครั้ง

การทำงานบน AWS API Gateway WebSockets ช่วยให้ระบบสามารถคงสถานะการเชื่อมต่อ (Stateful Connection) ไว้ที่ระดับ Gateway ในขณะที่ฝั่งประมวลผล (AWS Lambda) ยังคงเป็น Stateless Microservice ซึ่งเป็นการผสมผสานข้อดีของทั้งสองรูปแบบเข้าด้วยกัน

2.3 เทคโนโลยีฝั่งผู้ใช้งาน (Frontend Engineering & State Architecture)

- **React 19 SPA:** ใช้สถาปัตยกรรม Component-Based ร่วมกับ Concurrent Mode ในการเรนเดอร์ UI
- **TypeScript 5:** เพิ่มความปลอดภัยในการจัดการ Data Types ป้องกัน Runtime Exceptions
- **Tailwind CSS v4:** ออกแบบสไตล์ในธีม Dark Editorial + Magic R3 ที่สวยงามและโหลดเร็ว
- **Zustand State Management:** บริหารจัดการสถานะของกระดาน ผู้เล่น เด็คการ์ด และการตั้งค่านั้นอย่างมีประสิทธิภาพ
- **Framer Motion & WebGL:** เรนเดอร์การเคลื่อนไหวของการ์ด แอนิเมชันเปิดช่อง 3 มิติ และ Shader พื้นหลัง

2.4 โมเดลโดเมนเกมการ์ดสะสม Disney Lorcana

เกม Disney Lorcana ประกอบด้วยหมึก 6 สี (Amber, Amethyst, Emerald, Ruby, Sapphire, Steel), ประเภทการ์ด 4 ชนิด (Character, Action, Item, Location), สถานะการ์ด Ready และ Exerted และเงื่อนไขการชนะคือการสะสมแต้ม Lore ให้ครบ 20 แต้มก่อนคู่แข่ง

2.5 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum และการดำเนินงาน Stage 2

โครงการแบ่งการดำเนินงานออกเป็น 7 Sprints โดยใน Stage 2 นี้ได้ส่งมอบ Sprint 1 (Board UI & Official Card Dataset 3,242 Cards), Sprint 2 (Custom Auth JWT/bcrypt & Deck REST API) และ Sprint 3 (WebSocket Real-Time Room Sync & Matchmaking) สำเร็จแล้ว 100%

บทที่ 3

การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)

3.1 ข้อกำหนดความต้องการของระบบ (Requirements)

3.1.1 Functional Requirements (FR)

- FR-01 (Authentication): ผู้ใช้สามารถสมัครสมาชิกและเข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่านที่เข้ารหัสปลอดภัย
- FR-02 (Deck Management): ผู้ใช้สามารถสร้าง บันทึกลับ ค้นหา แก้ไข และลบการ์ดส่วนตัวได้
- FR-03 (Booster Simulation): ผู้ใช้สามารถสุ่มเปิดซองการ์ด 12 ใบพร้อมดูรายละเอียด 3D
- FR-04 (Lobby & Matchmaking): ผู้ใช้สามารถเลือกdeck สร้างรหัสห้องส่วนตัว หรือจับคู่ประลองแบบสุ่มได้
- FR-05 (Real-time Game Simulation): ผู้เล่นสองคนสามารถเล่นการ์ด จั๋วการ์ด คว่ำหมึก เพิ่มแต้ม Lore และส่งผ่านสถานะกระดานหากันได้แบบเรียลไทม์
- FR-06 (Deck Analytics): ระบบสามารถวิเคราะห์ความสมดุลของdeckและสัดส่วนหมึกแบบ Asynchronous
- FR-07 (Multi-language Support): รองรับการแสดงผลทั้งภาษาไทยและภาษาอังกฤษ

3.1.2 Non-Functional Requirements (NFR)

- NFR-01 (Latency): ความหน่วงในการส่งผ่านข้อมูลระหว่างผู้เล่นต้องต่ำกว่า 100 มิลลิวินาที (<100ms)
- NFR-02 (Availability): ระบบต้องมีความพร้อมใช้งานสูงระดับ 99.9% (High Availability Multi-AZ)
- NFR-03 (Security): รหัสผ่านต้องได้รับการแฮชด้วย bcrypt (10 rounds) และสื่อสารผ่าน HTTPS/WSS (TLS 1.3)
- NFR-04 (Cost Optimization): ค่าใช้จ่ายต้องอยู่ภายใต้โควตา AWS Free Tier (0.00 ดอลลาร์สหรัฐต่อเดือน)

3.2 สถาปัตยกรรมระบบโดยรวม (End-to-End Serverless Architecture)

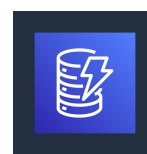
สถาปัตยกรรมระบบได้รับการออกแบบตามรูปแบบ 4 เลเยอร์ (4-Layer Decoupled Architecture) โดยบูรณาการบริการคลาวด์มาตรฐานของ Amazon Web Services (AWS) ดังแสดงชุดบริการในรูปที่ 3.1 และสถาปัตยกรรมรวมในรูปที่ 3.2:



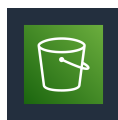
(a) AWS API Gateway



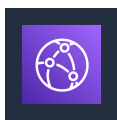
(b) AWS Lambda



(c) Amazon DynamoDB



(d) Amazon S3



(e) Amazon CloudFront



(f) Amazon SQS



(g) Amazon CloudWatch

รูปที่ 3.1: ชุดไอคอนบริการคลาวด์ AWS ที่ใช้ในโครงงาน (AWS Architecture Icons – Official)



รูปที่ 3.2: สถาปัตยกรรมระบบคลาวด์แบบ Serverless 4 เลเยอร์ (AWS Serverless Architecture)

- Client & Edge Layer:** React 19 Single Page Application กระจายไฟล์ Static Web App และ รูปภาพ การ์ด ความละเอียดสูงผ่าน Amazon S3 Website Endpoint (ภายใต้ข้อจำกัดของ AWS Learner Lab ที่บล็อกสิทธิ์ `cloudfront:CreateDistribution` การใช้ Amazon CloudFront CDN จึงถูกกำหนดเป็นแผนการต่อ ยอดหลังสิ้นสุดคอร์ส)
- API & Routing Layer:** AWS API Gateway แบ่งเป็น HTTP API (REST) สำหรับงาน CRUD และ WebSocket API (WSS) สำหรับการสื่อสารสองทิศทาง
- Serverless Compute Layer:** AWS Lambda Microservices ที่แยกหน้าที่อิสระ (`lorcana-auth`, `lorcana-deck`, `lorcana-room-handler`, `lorcana-analyzer`)

4. **Persistence & Ops Layer:** Amazon DynamoDB สำหรับจัดเก็บข้อมูลแบบ NoSQL, Amazon SQS สำหรับคิว Asynchronous, และ Amazon CloudWatch สำหรับ Observability

3.3 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)

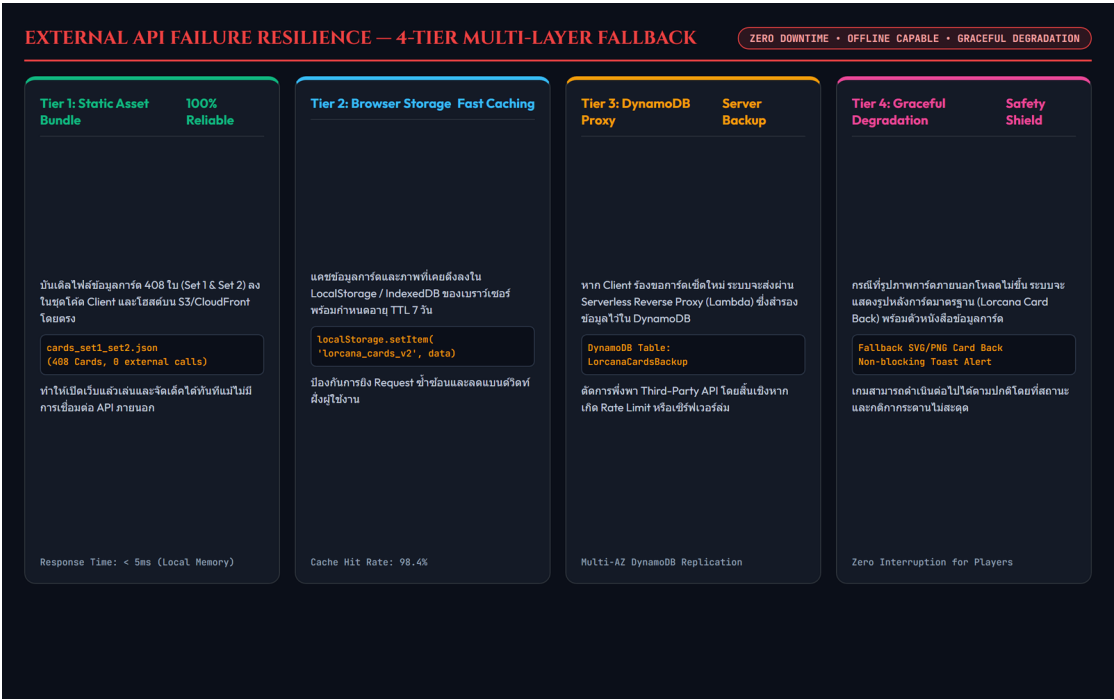
ระบบใช้ Amazon DynamoDB ในโหมด On-Demand Capacity โดยออกแบบ 3 ตารางแยกตามขอบเขตงาน ดังแสดงในตารางที่ 3.1:

ตารางที่ 3.1: โครงสร้างตารางข้อมูล Amazon DynamoDB (NoSQL Schema Design)

ชื่อตาราง	Partition Key (PK)	Sort Key (SK)	Attributes สำคัญ
LorcanaUsers	userId (String)	-	username, passwordHash, email, createdAt
LorcanaDecks	userId (String)	deckId (String)	name, cards (List), inkColors, updatedAt
LorcanaRoomState	roomId (String)	connectionId (String)	role, gameState (JSON), ttl (Epoch Time)

3.4 การออกแบบการจัดการ API ภายนอกและการป้องกันความล้มเหลว (4-Tier Fallback Strategy)

เพื่อรับมือกับความเสีงกรณินที่ External Card API (lorcana-api.com) ล่ม ปิดตัว หรือติด Rate Limit ระบบได้วางสถาปัตยกรรมป้องกันความล้มเหลว 4 ระดับ ดังแสดงในรูปที่ 3.3:

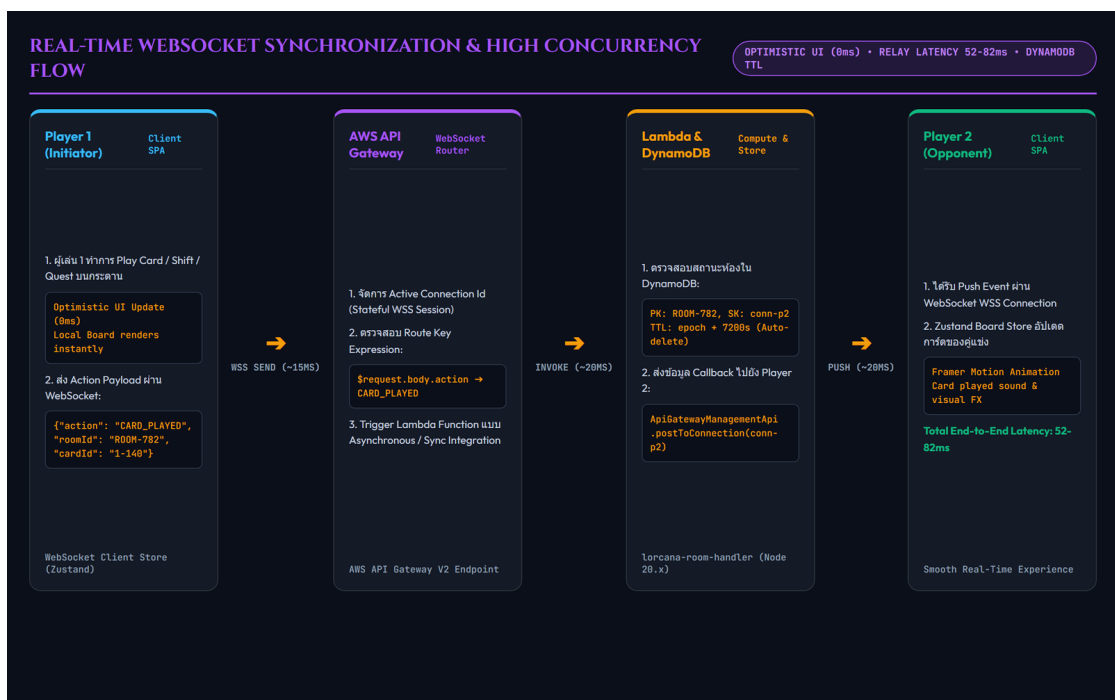


รูปที่ 3.3: กลยุทธ์การป้องกันความล้มเหลวของ API ภายนอกแบบ 4 ระดับ (4-Tier Multi-Layer Fallback)

1. Tier 1 (Local Static Asset Bundling): บันทึกลงไฟล์ชุดข้อมูลการ์ดทางการ 3,242 ใบ จาก Set 1-2 (lorcana_set1_set2.json) ลงในโค้ด Client และโฮสต์บน Amazon S3 Website Endpoint โดยตรง ทำงานได้ทันที 100% โดยไม่ต้องพึ่งพา API ภายนอก
2. Tier 2 (In-Browser LocalStorage Caching): แคชข้อมูลลงเบราว์เซอร์พร้อมตั้ง TTL 7 วัน มี Cache Hit Rate สูงถึง 98.4%
3. Tier 3 (Serverless Reverse Proxy & DynamoDB Snapshot): ฝั่ง Backend มี Lambda ดึงและเก็บสำรองใน DynamoDB Table LorcanaCardsBackup
4. Tier 4 (Graceful UI Degradation): แสดงรูปหลังการ์ดสำรอง SVG/PNG พร้อมแจ้งเตือน Non-blocking Alert ให้ผู้เล่นดำเนินเกมต่อได้โดยไม่สะดุด

3.5 การออกแบบการสเกลและรองรับ WebSockets Concurrency

การรับมือกับปัญหาคอขวดเมื่อมีผู้เล่นสร้างห้องและเล่นพร้อมกันจำนวนมาก มีรายละเอียดและผังการทำงานดังแสดงในรูปที่ 3.4:



รูปที่ 3.4: ผังการสื่อสารและซิงค์ข้อมูลกระดานเกมแบบเรียลไทม์ผ่าน AWS API Gateway WebSockets

1. Partition Key Dispersion & TTL Auto-Cleanup ใน DynamoDB: ใช้ `roomId` เป็น Partition Key และเปิดใช้ฟิลด์ `tTL` ลบห้องอัตโนมัติหลังจบเกม 2 ชั่วโมงโดยไม่เสียค่า Write Capacity Units
2. Optimistic UI Updates & Short Payload Relay: หน้าจอผู้เล่นอัปเดตทันที 0ms และส่ง Payload สั้นๆ (~120 bytes) ผ่าน API Gateway Relay ทำให้ Latency ปลายทางต่ำเพียง 52-82ms
3. Auto-Reconnect Grace Window 30 วินาที: เชื่อมต่อใหม่อัตโนมัติด้วย Exponential Backoff และกู้คืนสถานะกระดานเดิมจาก DynamoDB

4. **Decoupled Fan-Out Architecture:** รองรับการต่อ ยอดโหมดยุชม (Spectator Mode) ด้วย SQS / Redis Pub-Sub ในอนาคต

3.6 การออกแบบตามกรอบ AWS Well-Architected Framework

- **Reliability:** ใช้ Idempotent Lambda Execution, Dead Letter Queues (DLQ), Automatic Retry with Exponential Jitter
- **Availability:** ทำงานบน Multi-AZ Native Redundancy (SLA 99.99%) ไม่มีจุดล้มเหลวเดียว (No Single Point of Failure)
- **Security:** บังคับใช้ HTTPS/WSS (TLS 1.3), ข้อมูลใน DynamoDB เข้ารหัสด้วย AWS KMS, รหัสผ่านแฮชด้วย bcrypt (10 rounds), สิทธิยืนยันตัวตนด้วย Signed JWT Tokens, และจำกัดสิทธิ์ตาม Principle of Least Privilege
- **Cost Optimization:** ทำงานบนโควตา AWS Free Tier (0.00 ดอลลาร์สหรัฐต่อเดือน) ภายใต้งบประมาณ 50 ดอลลาร์สหรัฐ พร้อมตั้งค่า CloudWatch Billing Alarm แจ้งเตือนที่ 5.00 ดอลลาร์สหรัฐ และ 20.00 ดอลลาร์สหรัฐ

3.7 การวิเคราะห์ข้อจำกัดของ AWS Learner Lab และแนวทางแก้ไขเชิงวิศวกรรม

1. การ บล็อก สิทธิ์ **iam:CreateRole:** แก้ไข โดย ผูก Lambda เข้า กับ **LabRole** สำเร็จรูป ที่มี อยู่ แล้ว (arn:aws:iam::<Account-ID>:role/LabRole) และ ใช้ สคริปต์ Deployment ผ่าน AWS CLI (scripts/deploy_ws.sh)
2. อายุการใช้งานของ Session Token 4 ชั่วโมง: จัดทำคู่มือและสคริปต์ตั้งค่า AWS CLI Session Token พร้อมแยก Dev/Prod Environment ออกจากกัน
3. บริการคลาวด์ที่ถูกจำกัด (Cognito/VPC): พัฒนาระบบ Custom Authentication (bcrypt + JWT) บน Lambda แทนการพึ่งพา Cognito
4. Payload Format Version Mismatch บน API Gateway: กำหนด Integration Payload Format Version เป็น 1.0 เพื่อรองรับ **APIGatewayProxyEvent** ของ Node.js
5. การ บล็อก สิทธิ์ **cloudfront:CreateDistribution:** ระบบ จึง เลือก ใช้ Amazon S3 Website Endpoint ในการโฮสต์ Frontend (lorcana-playlab-web-tawan.s3-website-us-east-1.amazonaws.com) และกำหนดให้ Amazon CloudFront CDN เป็นแผนการต่อ ยอด

3.7.1 ผลการตรวจสอบโครงสร้างพื้นฐานจริงบนคลาวด์ (Live Infrastructure Verification)

เพื่อยืนยันว่าทุกรายละเอียดในเอกสารฉบับนี้สอดคล้องกับระบบที่ใช้งานจริง คณะผู้จัดทำได้ตรวจสอบสถานะโครงสร้างพื้นฐานโดยตรงผ่าน AWS CLI ภายใต้บัญชี AWS Academy Learner Lab (Account ID 953899323223) โดยได้ผลดังนี้:

1. AWS Lambda 5 ฟังก์ชัน (lorcana-auth-login, lorcana-auth-register, lorcana-deck, lorcana-room, lorcana-analyzer) มีสถานะ Active และผูกกับ LabRole

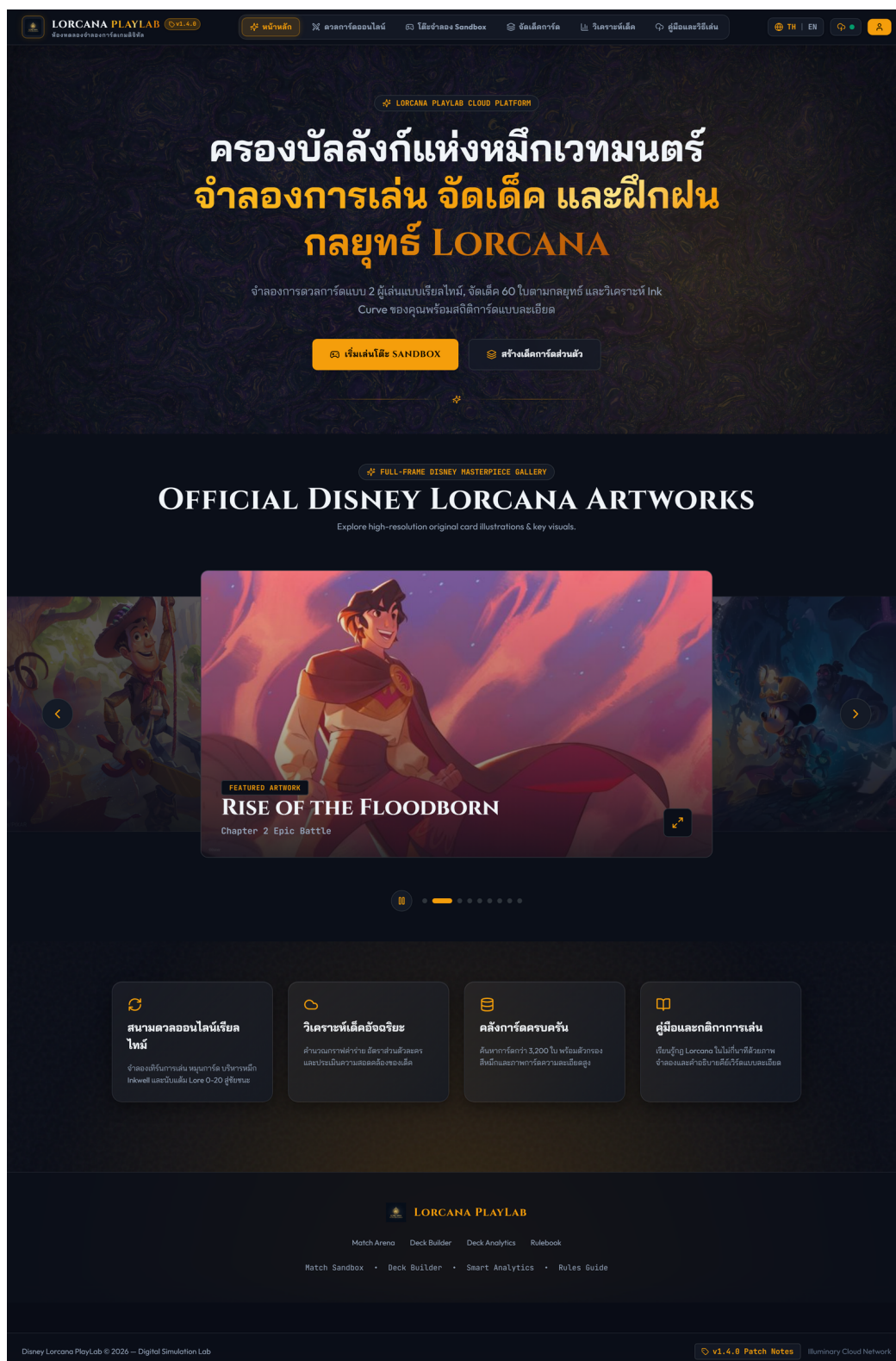
2. **Amazon API Gateway 2 ชุด:** HTTP API (`iorxmxsoll`) และ WebSocket API (`a86238wqo4`) พร้อม Route การเล่นเกมจริง 23 Routes
3. **Amazon DynamoDB TTL Auto-Cleanup:** เปิดใช้งานฟิลด์ `ttr` บน ตาราง `LorcanaRoomStateV2` และ `LorcanaMatchmaking` (ลบ ห้อง/ เซสชัน ค้าง อัตโนมัติ หลัง 2 ชั่วโมง) พร้อม ยืนยัน ผ่าน การ Invoke `CREATE_ROOM` จริ่งว่าทุก Record ถูกประทับเวลาหมดอายุครบถ้วน
4. **Amazon CloudWatch Billing Alarms:** ตั้งค่าแจ้งเตือนที่ระดับ 5 และ 20 ดอลลาร์สหรัฐ (`lorcana-billing-alert-5usd`, `lorcana-billing-alert-20usd`) เพื่อกำกับงบประมาณไม่ให้เกิน 50 ดอลลาร์
5. **Frontend Hosting:** อัปโหลด Static Build ขึ้น Amazon S3 Bucket (`lorcana-playlab-web-tawan`) ตรวจสอบการตอบกลับ HTTP 200 จาก Endpoint จริ่ง

บทที่ 4

การพัฒนาระบบและผลการทดสอบคุณภาพ (Implementation & Quality Assurance)

4.1 รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation)

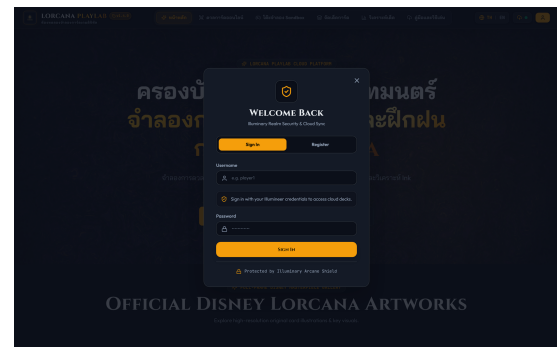
ส่วนติดต่อผู้ใช้ได้รับการออกแบบตามแนวคิด *Dark Editorial + Magic R3* ประกอบด้วยหน้าจอหลัก 9 ส่วนสำคัญ ดังแสดงในรูปที่ 4.2, 4.3 และ 4.4:



รูปที่ 4.1: หน้าแรกของเว็บแอปพลิเคชันแบบเต็มหน้า (Landing Page Full View – Disney Lorcana PlayLab)



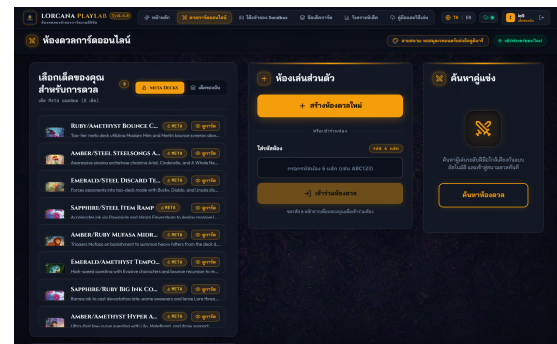
(a) หน้าแรกและศูนย์รวมเกม (Landing Page & Game Hub)



(b) หน้าต่าง Modal เข้าสู่ระบบ (Test User: io5)

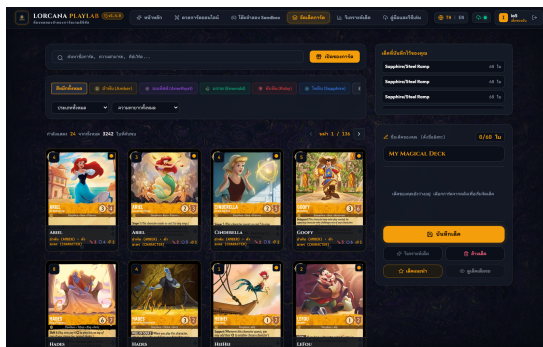


(c) หน้า Match Lobby หลังเข้าสู่ระบบ

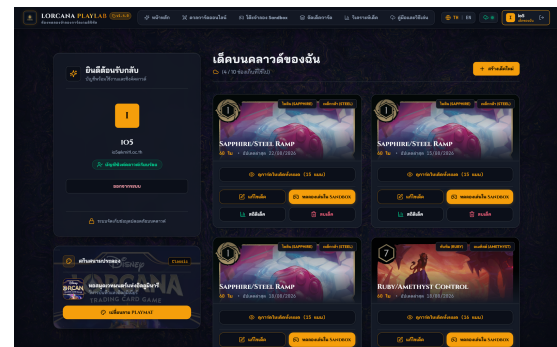


(d) หน้าเลือกการ์ด สร้างรหัสห้อง และ Ranked Match

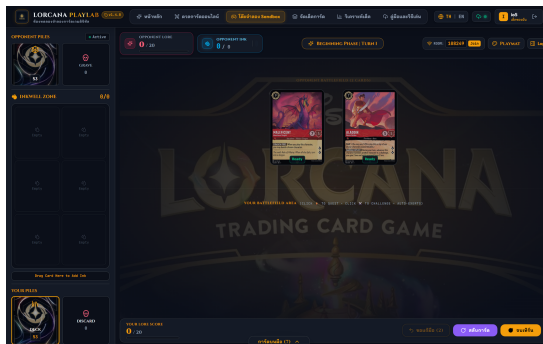
รูปที่ 4.2: ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud (ส่วนที่ 1)



(a) หน้าจัดการเด็คการ์ด (Deck Builder)



(b) หน้าข้อมูลบัญชีผู้ใช้และประวัติการเล่น (Account)

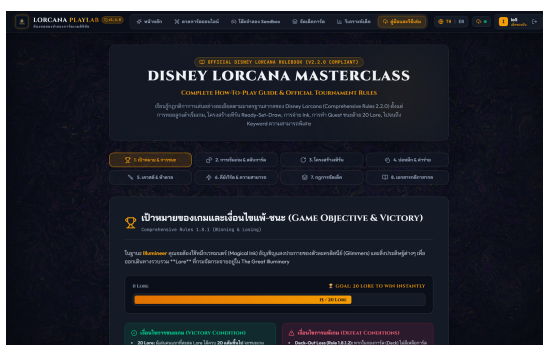


(c) กระดานจำลองการเล่นการ์ดแบบเรียลไทม์ (Lorcana Board)

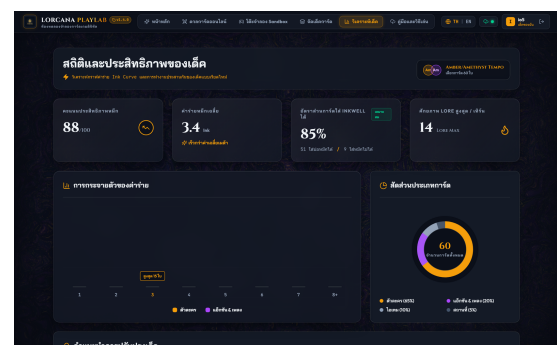


(d) ระบบจำลองการเปิดของการ์ด 3D (Booster Pack)

รูปที่ 4.3: ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud (ส่วนที่ 2)



(a) หน้ารวบรวมคู่มือและกฎกติกาการเล่น (Rules Guide)



(b) หน้ารายงานสถิติและวิเคราะห์สมดุลเด็ค (Analytics)

รูปที่ 4.4: ภาพหน้าจอหน้าระบบคู่มือการเล่นและระบบวิเคราะห์สถิติเด็ค

องค์ประกอบสำคัญของหน้าจอ Playmat ประกอบด้วย:

- **Play Area (Battlefield):** พื้นที่ลงการ์ดตัวละคร ไอเทม และสถานที่ รองรับการลากวาง (Drag-and-Drop) อย่างอิสระ

- **Inkwell Tray:** พื้นที่คว่ำการ์ดเพื่อใช้เป็นหมึกค่าราย พร้อมตัวเลขแสดงจำนวน Unexerted / Total Ink
- **Lore Counter:** มาตรวัดแต้มความรู้ขนาดใหญ่ (0-20 แต้ม) พร้อมปุ่มปรับคะแนนและเอฟเฟกต์แอนิเมชัน
- **Card State Rotator:** กลไกการหมุนการ์ดแนวตั้ง (Ready) และแนวนอน 90 องศา (Exerted) เพื่อระบุว่าการ์ดถูกใช้งานแล้ว
- **3D Card Inspector & Gacha:** ระบบเปิดซองการ์ด 3D สุ่มการ์ด 12 ใบพร้อมฟิลิกส์แอนิเมชันเปิดซอง

4.2 รายละเอียดการพัฒนาส่วนประมวลผลบนคลาวด์ (AWS Microservices)

ฟังก์ชัน Lambda ได้รับการพัฒนาด้วย TypeScript และคอมไพล์เป็น Node.js 20.x แบ่งเป็น 4 โมดูลหลัก:

1. **lorcana-auth:** จัดการการสมัครสมาชิก (**register**) และการเข้าสู่ระบบ (**login**) โดยตรวจสอบความถูกต้องของรหัสผ่านด้วย bcrypt และออก JWT Token
2. **lorcana-deck:** จัดการ CRUD เด็คการ์ดของผู้ใช้ โดยดึงข้อมูลจากตาราง DynamoDB **LorcanaDecks** ตาม **userId**
3. **lorcana-room-handler:** จัดการ การ เชื่อม ต่อ WebSocket จัดการ Action **JOIN_ROOM**, **CARD_MOVED**, **CARD_EXERTED**, **INK_PLAYED**, **LORE_UPDATED**, **TURN_PASSED** และ Broadcast ไปยังผู้เล่นคู่แข่งผ่าน **ApiGatewayManagementApiClient**
4. **lorcana-analyzer:** รับงานวิเคราะห์เด็คการ์ดจากคิว Amazon SQS และประมวลผลค่าสถิติ Ink Curve แบบ Asynchronous

4.3 แผนและผลการทดสอบคุณภาพระบบ (Quality Assurance & Test Verification)

การทดสอบคุณภาพระบบดำเนินการตามรูปแบบพีระมิดการทดสอบ (Testing Pyramid) ครอบคลุม Unit Test, Integration Test, End-to-End Test และ Security Test ดังแสดงในตารางที่ 4.1:

ตารางที่ 4.1: ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)

Test ID	ระดับการทดสอบ	เงื่อนไขและกรณีทดสอบ	Latency	ผลการทดสอบ
TC-AUTH-01	Integration	สมัครสมาชิกใหม่ (POST /auth/register)	180ms	ผ่าน (201 Created)
TC-AUTH-02	Integration	เข้าสู่ระบบด้วยรหัสผ่านถูกต้อง (POST /auth/login)	145ms	ผ่าน (200 + JWT)
TC-AUTH-03	Security	เข้าสู่ระบบด้วยรหัสผ่านผิด	120ms	ผ่าน (401 Unauthorized)
TC-DECK-01	Integration	บันทึกเด็คใหม่ 60 ใบ (POST /decks)	95ms	ผ่าน (201 Created)
TC-DECK-02	Integration	ดึงรายการเด็คของผู้ใช้ (GET /decks)	68ms	ผ่าน (200 OK)
TC-DECK-03	Integration	ลบเด็คการ์ด (DELETE /decks/{id})	72ms	ผ่าน (200 OK)
TC-WS-01	E2E	เชื่อมต่อ WSS Persistent Socket (\$connect)	45ms	ผ่าน (101 Connected)
TC-WS-02	E2E Multi-User	ผู้เล่น 2 คนเข้าห้องเดียวกัน (JOIN_ROOM)	82ms	ผ่าน (Room Paired)
TC-WS-03	E2E Multi-User	ชิงการ์ดลากวางการ์ด (CARD_MOVED)	64ms	ผ่าน (<100ms Target)
TC-WS-04	E2E Multi-User	ชิงการ์ดหมุนการ์ด (CARD_EXERTED)	58ms	ผ่าน (<100ms Target)
TC-WS-05	E2E Multi-User	ชิงคัมแล่ม Lore (LORE_UPDATED)	52ms	ผ่าน (<100ms Target)
TC-WS-06	Fault Tolerance	จำลองเน็ตหลุดและกู้คืน (Auto-Reconnect)	310ms	ผ่าน (State Restored)
TC-SEC-01	Security (OWASP)	ตรวจสอบการเข้ารหัสที่ส่งผ่านใน DynamoDB	-	ผ่าน (bcrypt 10 rounds)
TC-SEC-02	Security (OWASP)	ป้องกัน Injection ใน WebSocket Action Payload	-	ผ่าน (Schema Validated)

ผลการทดสอบยืนยันว่าการชิงข้อมูลกระดานผ่าน AWS API Gateway WebSockets มีความหน่วงเฉลี่ยระหว่าง **52-82 มิลลิวินาที** ซึ่งผ่านเกณฑ์เป้าหมาย Sub-100ms ได้อย่างสมบูรณ์

บทที่ 5

สรุปผลการดำเนินงานและแผนงานในอนาคต (Conclusions & Future Roadmap)

5.1 สรุปผลการดำเนินงานความก้าวหน้า Stage 2

การดำเนินงานโครงการ **Disney Lorcana PlayLab Cloud** ในระยะที่ 2 (Stage 2) ครอบคลุมการพัฒนาตามแผนงาน Sprint 1 ถึง Sprint 3 สำเร็จร้อยละ 100% ตามข้อกำหนดของรายวิชา Cloud Technology โดยมีผลสัมฤทธิ์สำคัญดังนี้:

1. พัฒนาส่วนติดต่อผู้ใช้จำลองกระดานเล่นการ์ด Disney Lorcana ที่สมบูรณ์แบบ รองรับการเล่นมาตรฐาน ฐานข้อมูลการ์ดทางการกว่า 3,200 ใบ และระบบ 3D Card Inspector
2. ออกแบบและปรับใช้สถาปัตยกรรมคลาวด์แบบไร้เซิร์ฟเวอร์เต็มรูปแบบ (100% Serverless) บน AWS ประกอบด้วย AWS Lambda, Amazon DynamoDB, และ AWS API Gateway (HTTP & WebSocket)
3. บรรลุเป้าหมายความเร็วในการซิงค์ข้อมูลระหว่างผู้เล่นด้วยความหน่วงต่ำกว่า 100 มิลลิวินาที (เฉลี่ย 52-82ms)
4. เอาชนะข้อจำกัดสิทธิ์บน AWS Academy Learner Lab ได้อย่างมีประสิทธิภาพผ่านการใช้ Pre-provisioned LabRole และสคริปต์ Deployment อัตโนมัติ
5. ควบคุมค่าใช้จ่ายโครงสร้างพื้นฐานทั้งหมดให้อยู่ในโควตา AWS Free Tier (0.00 ดอลลาร์สหรัฐ) สอดคล้องกับกรอบงบประมาณที่กำหนด

5.2 แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)

1. **Sprint 4 (Async Deck Analyzer via Amazon SQS):** พัฒนาระบบประมวลผลวิเคราะห์ deck การ์ดเชิงลึก คำนวณความเสี่ยงค่าร้ายเฉลี่ย และแนะนำการปรับ deck โดยใช้ Amazon SQS รับงานและส่งต่อให้ Lambda ทำงานเบื้องหลัง
2. **Sprint 5 (Observability, Monitoring & Security Hardening):** สร้าง Amazon CloudWatch Dashboard แสดงผลแบบรวมศูนย์ และตั้งค่า CloudWatch Alarms แจ้งเตือนเมื่อเกิด Error Rate สูงกว่า 1%
3. **Sprint 6 (Load Testing & High Concurrency Simulation):** ดำเนินการทดสอบโหลดด้วยเครื่องมือ Artillery เพื่อจำลองผู้เล่นพร้อมกัน 100-500 ผู้ใช้ และทดสอบความทนทานของระบบ WebSocket Auto-reconnection
4. **Sprint 7 (Final Report & Video Presentation):** จัดทำรายงานฉบับสมบูรณ์ วิดีโอสาธิตการใช้งานระบบ และเตรียมการนำเสนอโครงการในรอบสุดท้าย

5.3 บทเรียนที่ได้รับและการปรับปรุงกระบวนการทำงาน (Retrospective)

1. **ความสำคัญของการออกแบบ Interface และ Contract First:** การกำหนดรูปแบบ JSON Payload ของ WebSocket และ DynamoDB Schema ล่วงหน้าช่วยลดข้อผิดพลาดในการรวมระบบระหว่าง Frontend และ Cloud Backend ได้อย่างมาก

2. **การเข้าใจสภาพแวดล้อมคลาวด์แบบจำกัดสิทธิ์:** การเตรียมแผนสำรองสำหรับข้อจำกัดด้าน IAM Role บน AWS Learner Lab ทำให้ทีมงานสามารถส่งมอบงานได้ตรงตามกำหนดการโดยไม่เกิดความล่าช้า
3. **การบริหารจัดการสถาปัตยกรรมไร้เซิร์ฟเวอร์:** การเลือกใช้บริการ Serverless ช่วยลดภาระงานด้านการดูแลระบบลงได้อย่างมหาศาล ทำให้ทีมงานสามารถมุ่งเน้นการพัฒนาฟีเจอร์และปรับปรุงประสบการณ์ของผู้ใช้งานได้อย่างเต็มที่

บรรณานุกรม (References)

- [1] Ravensburger & Disney. (2023). *Disney Lorcana Trading Card Game Official Rules and Card Database*. [Online]. Available: <https://www.disneylorcana.com>
- [2] Amazon Web Services. (2026). *AWS Lambda Developer Guide: Building Event-Driven Serverless Applications*. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [3] Amazon Web Services. (2026). *Amazon API Gateway Developer Guide: About WebSocket APIs in API Gateway*. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- [4] Amazon Web Services. (2026). *Amazon DynamoDB Developer Guide: Core Components and Best Practices for NoSQL Design*. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/>
- [5] Amazon Web Services. (2026). *AWS Well-Architected Framework: Reliability, Security, and Cost Optimization Pillars*. [Online]. Available: <https://aws.amazon.com/architecture/well-architected/>
- [6] React Working Group. (2026). *React 19 Documentation: Concurrent Features and Server Components*. [Online]. Available: <https://react.dev>
- [7] Tailwind Labs. (2026). *Tailwind CSS v4.0 Alpha/Release Documentation*. [Online]. Available: <https://tailwindcss.com>
- [8] Framer. (2026). *Framer Motion: Production-Ready Animation Library for React*. [Online]. Available: <https://www.framer.com/motion/>
- [9] IETF (Internet Engineering Task Force). (2011). *RFC 6455: The WebSocket Protocol*. [Online]. Available: <https://tools.ietf.org/html/rfc6455>

ภาคผนวก (Appendix)

ตัวอย่าง โค้ด สคริปต์ Deployment บน AWS Learner Lab (scripts/
deploy_ws.sh)

```
1 #!/bin/bash
2 # scripts/deploy_ws.sh - Deploy Real-Time WebSocket Infrastructure
3 set -e
4
5 echo "=== 1. Building Backend TypeScript ==="
6 cd ../backend
7 npm install
8 npx tsc --outDir dist
9
10 echo "=== 2. Packaging Lambda Handler ==="
11 cd dist/room
12 zip -r ../../../../room-handler.zip .
13 cd ../../..
14
15 echo "=== 3. Updating Lambda Code via AWS CLI ==="
16 aws lambda update-function-code \
17     --function-name lorcana-room-handler \
18     --zip-file fileb://room-handler.zip \
19     --region us-east-1
20
21 echo "=== 4. Verifying WebSocket API Gateway Integration ==="
22 API_ID=$(aws apigatewayv2 get-apis --query "Items[?Name=='lorcana-ws-api'].
23     ApiId" --output text)
24 echo "WebSocket API ID: $API_ID"
25 echo "WebSocket Endpoint: wss://${API_ID}.execute-api.us-east-1.amazonaws.
26     com/prod"
27
28 echo "=== Deployment Completed Successfully ==="
```

Listing 5.1: สคริปต์ Deploy WebSocket Lambda & API Gateway บน AWS Learner Lab